Università degli Studi di Torino

Dipartimento di Informatica



TECNOLOGIE DEL LINGUAGGIO NATURALE

HIRING PADAWAN

Budaes Tanaru Riccardo Gabriele

Driza Andrea

Indice

1. Introduzione
2. Codice
3. Eliza
4. Conversazioni
5. Analisi degli errori
6. Fenomeni linguistici gestiti
7. SpeechRecognition e Text2Speech
    1. Analisi degli errori
8. Conclusioni
9. Sitografia

1 Introduzione

Il progetto consiste nella creazione di un Dialog System (DS), ovvero un sistema di dialogo il cui scopo è interrogare l'utente sulla sua conoscenza della cultura Jedi. Impersonando il personaggio di Darth Vader o di Obi-Wan Kenobi, il sistema valuterà se l'utente sia in grado di diventare un Padawan Jedi o un Sith. L'implementazione è suddivisa in moduli, rispettivamente

1. **frame.py**

   Il modulo è la struttura d'informazione del DS, in particolare contiene un identificatore principale il quale permette la distinzione tra *'jedi* e *'sith'*. In base alla scelta del 'lato' da cui stare, è possibile trovare rispettivamente nome del personaggio intervistante, frasi d'interazione d'inizio e fine test ma anche di risposta corretta/errata. Il sistema associa a ogni domanda delle parole chiave (keyword) utili a verificare la correttezza della risposta dell'utente.

2. **bot.py**

   Il modulo contiene la logica principale del dialog system, definisce la classe responsabile del flusso d'interazione. All'inizio dell'interazione il bot inizializza le strutture che permettono di tenere traccia di ciò che accade durante la conversazione. Le risposte fornite dall'utente vengono registrate e ad ogni domanda viene associato l'esisto e la misura di affidabilità della risposta data.

   Ogni risposta fornita subisce un'analisi, dove si cerca di valutare l'accetabilità della stessa. Le domande vengono pescate randomicamente e senza ripetizione, reso possibile grazie al mantenimento in memoria di quelle che sono state le domande già poste. Per ogni domanda che viene espressa, il ds riconosce il tipo di risposta associata ovvero domanda a risposta multipla, singola o vero/falso gestendo la risposta in maniera coerente. Per una valutazione corretta tutte le risposte dell'utente vengono normalizzate trasformando quindi l'input in minuscolo e rimuovendo eventuali accenti finali o intermedi presenti nelle parole. Per quanto riguarda le domande a risposta binaria (*vero/falso*) le varianti positive/negative vengono mappate per le lingue d'interesse ovvero italiano e inglese.

   Ad ogni domanda viene inoltre fornito un massimo di tentativi possibili pari a tre; qui il dialog system decide in maniera del tutto casuale se fermarsi dal riproporre la domanda nel caso di risposta errata.

   Al termine del quiz quindi quando non ci sono più domande da proporre, viene calcolato il punteggio dell'utente. Dal punteggio risultante viene detto all'utente se sia riuscito o meno a diventare padawan del lato voluto con associato anche il numero di risposte correttamente date.

2 Codice



```python
def __init__(self, role, language):
        self.current_frame = frames[role]
        self.language = language
        self.memory = []
        self.common_ground = []
        self.asked_questions = []
        self.current_question = None
        self.current_question_frame = None

        self.questions = self.current_frame[f"questions_{language}"]
        self.good_responses = self.current_frame[f"good_responses_{language}"]
        self.bad_responses = self.current_frame[f"bad_responses_{language}"]
```

Figura 1: Inizializzazione

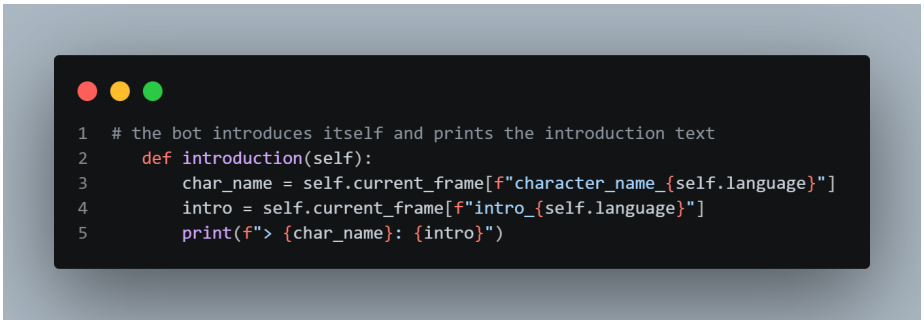
Costruttore della classe `FrameBot`. Ha il compito di inizializzare tutti gli attributi necessari al funzionamento del chatbot.

Parametri

- `role`: identifica il frame conversazionale da caricare dal dizionario `frames`.
- `language`: specifica la lingua utilizzata dal chatbot.

Funzionamento

- Carica il frame corrispondente al ruolo selezionato.
- Inizializza le strutture dati utilizzate durante l'interazione:
  - `memory`: memorizza tutti gli input dell'utente.
  - `common_ground`: conserva le informazioni condivise emerse durante il dialogo.
  - `asked_questions`: tiene traccia delle domande già poste.
- Inizializza le variabili relative alla domanda corrente.
- Carica le domande e le risposte associate alla lingua selezionata.

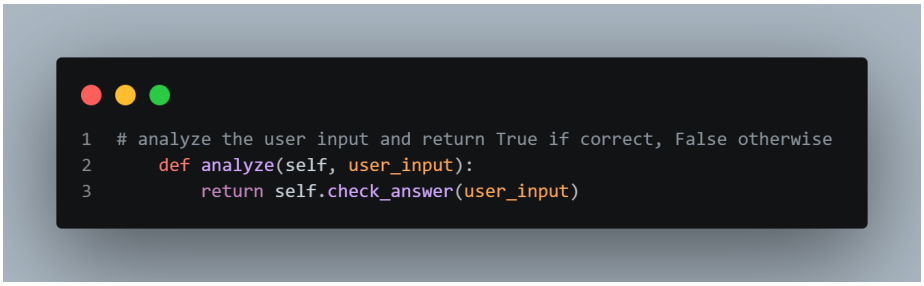


```python
# the bot introduces itself and prints the introduction text
    def introduction(self):
        char_name = self.current_frame[f"character_name_{self.language}"]
        intro = self.current_frame[f"intro_{self.language}"]
        print(f"> {char_name}: {intro}")
```

Figura 2: Introduzione

Mostra il messaggio introduttivo del chatbot.
Funzionamento

- Recupera il nome del personaggio dal frame corrente.
- Recupera il testo introduttivo nella lingua selezionata.
- Stampa il messaggio di presentazione.



```python
# analyze the user input and return True if correct, False otherwise
    def analyze(self, user_input):
        return self.check_answer(user_input)
```

Figura 3: Analizzatore risposta

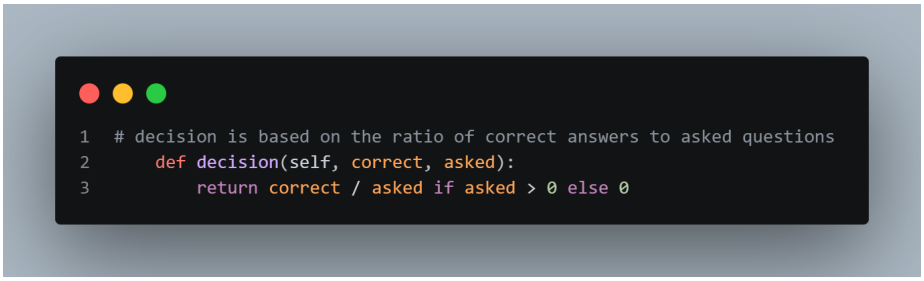
Analizza la risposta fornita dall'utente.
Parametro

- `user_input`: risposta inserita dall'utente.

Valore restituito

- `True` se la risposta è corretta.
- `False` in caso contrario.

Descrizione
La funzione delega il controllo della risposta al metodo `check_answer()`.



```python
# decision is based on the ratio of correct answers to asked questions
    def decision(self, correct, asked):
        return correct / asked if asked > 0 else 0
```

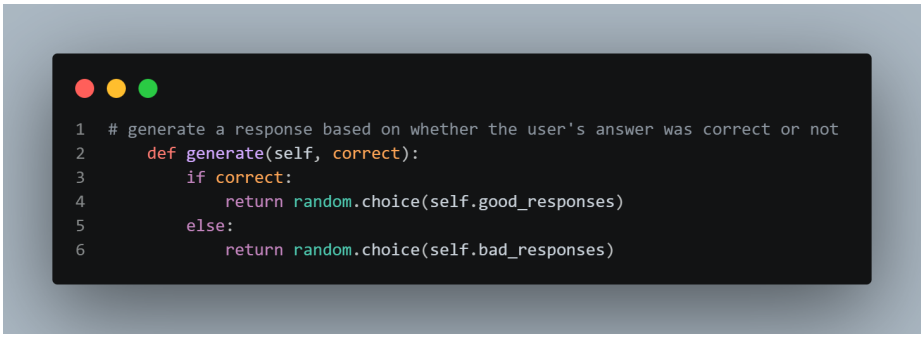
Figura 4: Decision

Calcola il rapporto tra risposte corrette e il numero totale di domande.
Parametri

- `correct`: numero di risposte corrette.
- `asked`: numero totale di domande poste.

Valore restituito

- Il rapporto `correct / asked`.
- Restituisce `0` se nessuna domanda è stata ancora posta.



```python
# generate a response based on whether the user's answer was correct or not
    def generate(self, correct):
        if correct:
            return random.choice(self.good_responses)
        else:
            return random.choice(self.bad_responses)
```

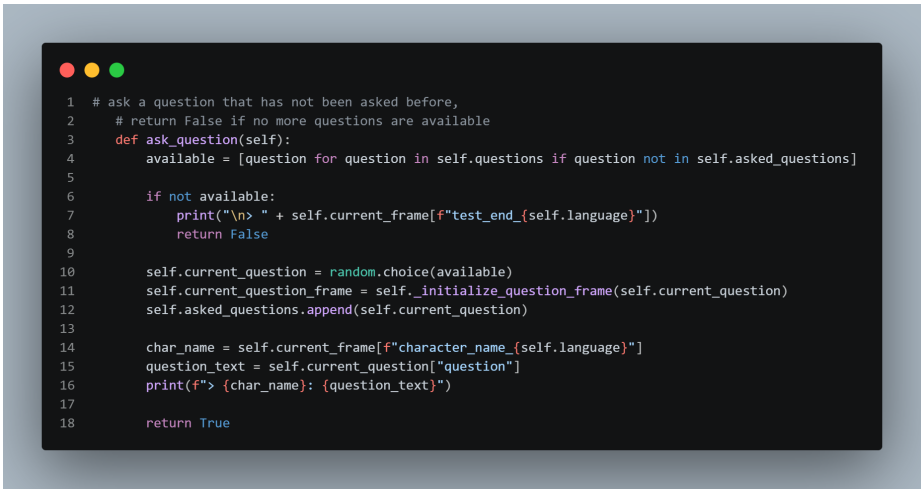
Figura 5: Generate

Genera un messaggio di feedback in base all'esito della risposta.
Parametro

- `correct`: valore booleano che indica se la risposta è corretta.

Valore restituito

- Una risposta positiva scelta casualmente se l'utente ha risposto correttamente.
- Una risposta negativa scelta casualmente in caso di errore.



```python
# ask a question that has not been asked before,
    # return False if no more questions are available
    def ask_question(self):
        available = [question for question in self.questions if question not in self.asked_questions]

        if not available:
            print("\n> " + self.current_frame[f"test_end_{self.language}"])
            return False

        self.current_question = random.choice(available)
        self.current_question_frame = self._initialize_question_frame(self.current_question)
        self.asked_questions.append(self.current_question)

        char_name = self.current_frame[f"character_name_{self.language}"]
        question_text = self.current_question["question"]
        print(f"> {char_name}: {question_text}")

        return True
```

Figura 6: Ask question

Seleziona e pone una nuova domanda all'utente.
Funzionamento

- Individua le domande non ancora utilizzate.
- Se non sono disponibili ulteriori domande, mostra il messaggio di fine test.
- Seleziona casualmente una domanda.
- Inizializza il frame associato alla domanda.
- Registra la domanda tra quelle già poste.
- Mostra il testo della domanda.

Valore restituito

- `True` se una nuova domanda è stata posta.
- `False` se non ci sono più domande disponibili.



```python
# build the frame for the current question and initialize its slots
    def _initialize_question_frame(self, question):
        expected_answers = list(question["answers"])

        if question.get("type") == "multi":
            slots = {
                "expected": expected_answers,
                "given": [],
                "result": None,
                "trials": 0,
                "confidence": 0.0,
                "matched": {answer: False for answer in expected_answers},
            }
        else:
            slots = {
                "expected": expected_answers,
                "given": None,
                "result": None,
                "trials": 0,
                "confidence": 0.0,
            }

        return {
            "question": question["question"],
            "type": question.get("type", "single"),
            "slots": slots,
        }
```

Figura 7: Inizializzazione frame domanda

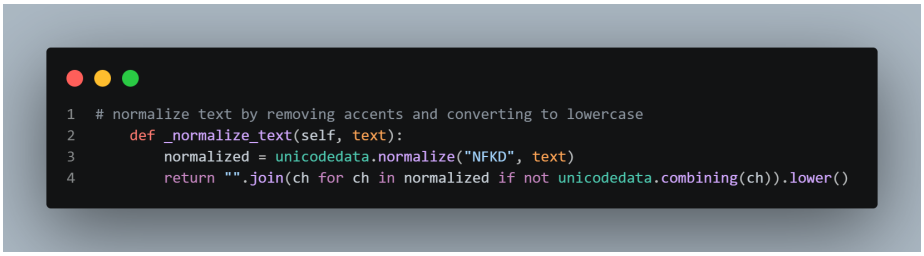
Costruisce la struttura dati necessaria per gestire una domanda.
Parametro

- `question`: dizionario contenente le informazioni sulla domanda.

Funzionamento

- Estrae le risposte attese.
- Crea gli slot necessari per monitorare le risposte fornite dall'utente.
- Gestisce separatamente domande a risposta singola e multipla.
- Inizializza campi come numero di tentativi, risultato e livello di confidenza.

Valore restituito
Restituisce un dizionario che rappresenta il frame della domanda corrente.



```python
# normalize text by removing accents and converting to lowercase
    def _normalize_text(self, text):
        normalized = unicodedata.normalize("NFKD", text)
        return "".join(ch for ch in normalized if not unicodedata.combining(ch)).lower()
```

Figura 8: Normalizzazione testo

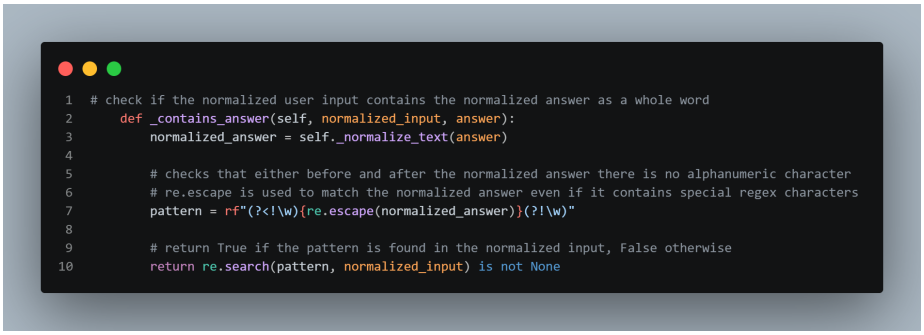
Normalizza una stringa per facilitarne il confronto.
Parametro

- `text`: testo da normalizzare.

Funzionamento

- Converte i caratteri accentati nelle rispettive forme base.
- Elimina i segni diacritici. (´, ‘, ^, ¸)
- Converte tutti i caratteri in minuscolo.

Valore restituito
Una stringa normalizzata.



```python
# check if the normalized user input contains the normalized answer as a whole word
    def _contains_answer(self, normalized_input, answer):
        normalized_answer = self._normalize_text(answer)

        # checks that either before and after the normalized answer there is no alphanumeric character
        # re.escape is used to match the normalized answer even if it contains special regex characters
        pattern = rf"(?<!\w){re.escape(normalized_answer)}(?!\w)"

        # return True if the pattern is found in the normalized input, False otherwise
        return re.search(pattern, normalized_input) is not None
```

Figura 9: Contains answer

Verifica se una determinata risposta è contenuta nell'input dell'utente.
Parametri

- `normalized_input`: input dell'utente già normalizzato.
- `answer`: risposta attesa.

Funzionamento

- Normalizza la risposta attesa.

- Costruisce un'espressione regolare che ricerca la risposta come parola completa.
- Evita corrispondenze parziali all'interno di altre parole.

Valore restituito

- `True` se la risposta è presente.
- `False` altrimenti.



```python
# map user input and expected labels to canonical true/false values
    def _to_bool_label(self, text):
        normalized = self._normalize_text(text).strip()

        true_aliases = {"vero", "true", "v", "t"}
        false_aliases = {"falso", "false", "f"}

        for token in true_aliases:
            if self._contains_answer(normalized, token):
                return "true"

        for token in false_aliases:
            if self._contains_answer(normalized, token):
                return "false"

        return None
```

Figura 10: To bool label

Converte diverse forme testuali in un'etichetta booleana standard.
Parametro

- `text`: testo da analizzare.

Funzionamento

- Riconosce varianti associate al valore vero.
- Riconosce varianti associate al valore falso.
- Converte tutte le forme riconosciute nelle etichette standard `"true"` e `"false"`.

Valore restituito

- `"true"`.
- `"false"`.
- `None` se non viene identificata alcuna corrispondenza.



Figura 11: Is true false question

Determina se una domanda appartiene alla categoria vero/falso.
Parametro

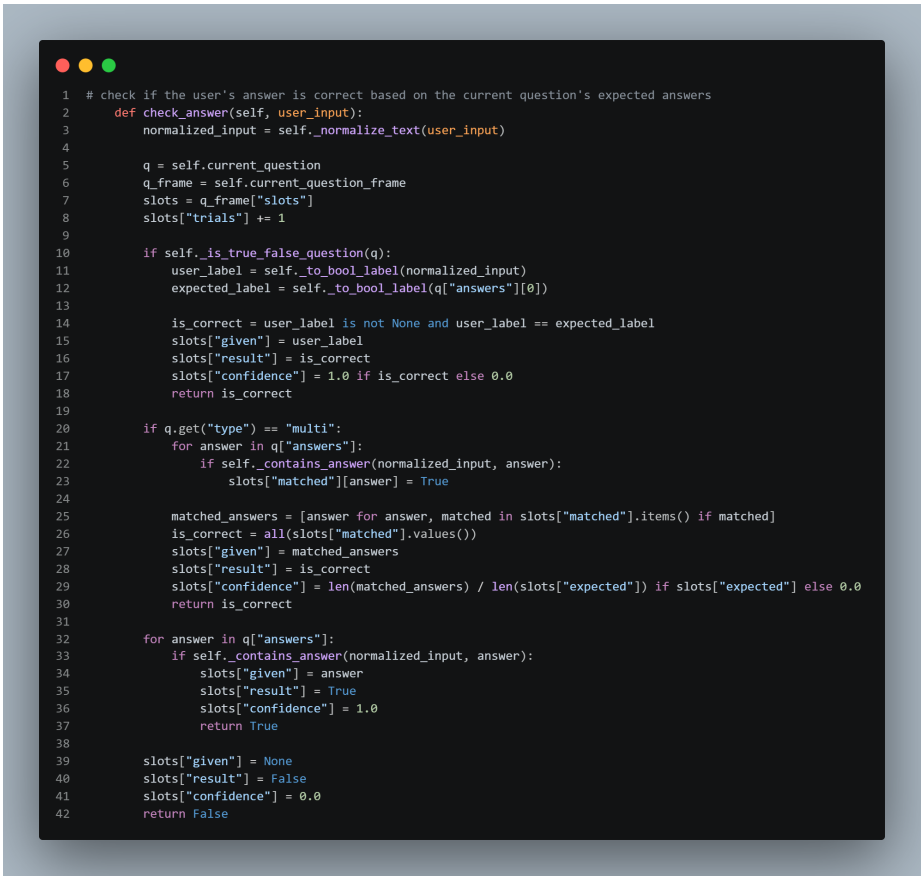
- `question`: dizionario contenente la domanda.

Funzionamento

- Converte le risposte attese in etichette booleane.
- Elimina eventuali valori non validi.
- Verifica se tutte le risposte appartengono alla stessa categoria booleana.

Valore restituito

- `True` se la domanda è di tipo vero/falso.
- `False` altrimenti.



```python
# check if the user's answer is correct based on the current question's expected answers
    def check_answer(self, user_input):
        normalized_input = self._normalize_text(user_input)

        q = self.current_question
        q_frame = self.current_question_frame
        slots = q_frame["slots"]
        slots["trials"] += 1

        if self._is_true_false_question(q):
            user_label = self._to_bool_label(normalized_input)
            expected_label = self._to_bool_label(q["answers"][0])

            is_correct = user_label is not None and user_label == expected_label
            slots["given"] = user_label
            slots["result"] = is_correct
            slots["confidence"] = 1.0 if is_correct else 0.0
            return is_correct

        if q.get("type") == "multi":
            for answer in q["answers"]:
                if self._contains_answer(normalized_input, answer):
                    slots["matched"][answer] = True

            matched_answers = [answer for answer, matched in slots["matched"].items() if matched]
            is_correct = all(slots["matched"].values())
            slots["given"] = matched_answers
            slots["result"] = is_correct
            slots["confidence"] = len(matched_answers) / len(slots["expected"]) if slots["expected"] else 0.0
            return is_correct

        for answer in q["answers"]:
            if self._contains_answer(normalized_input, answer):
                slots["given"] = answer
                slots["result"] = True
                slots["confidence"] = 1.0
                return True

        slots["given"] = None
        slots["result"] = False
        slots["confidence"] = 0.0
        return False
```

Figura 12: Check answer

Valuta la correttezza della risposta fornita dall'utente.
Parametro

- `user_input`: risposta dell'utente.

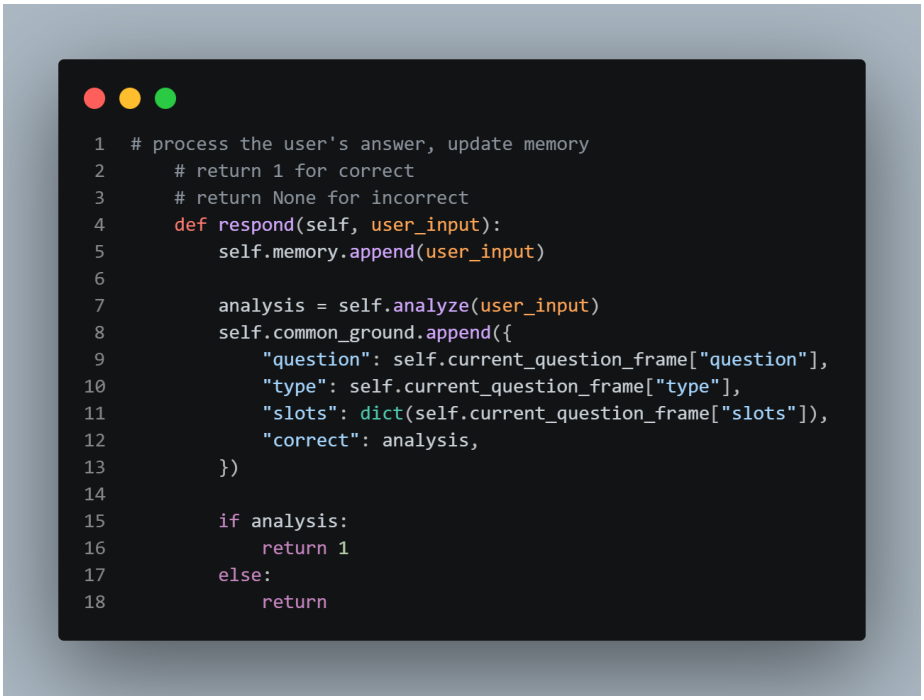
Funzionamento

- Normalizza l'input.
- Aggiorna il numero di tentativi effettuati.
- Gestisce tre casi distinti:
  1. Domande vero/falso.
  2. Domande a risposta multipla.
  3. Domande a risposta singola.

- Aggiorna gli slot con:
  - la risposta fornita;
  - il risultato della valutazione;
  - il livello di confidenza.

Valore restituito

- `True` se la risposta è corretta.
- `False` in caso contrario.



```python
# process the user's answer, update memory
    # return 1 for correct
    # return None for incorrect
    def respond(self, user_input):
        self.memory.append(user_input)

        analysis = self.analyze(user_input)
        self.common_ground.append({
            "question": self.current_question_frame["question"],
            "type": self.current_question_frame["type"],
            "slots": dict(self.current_question_frame["slots"]),
            "correct": analysis,
        })

        if analysis:
            return 1
        else:
            return
```

Figura 13: Respond

Gestisce la risposta dell'utente e aggiorna le strutture di memoria.
Parametro

- `user_input`: risposta fornita dall'utente.

Funzionamento

- Memorizza l'input dell'utente.
- Analizza la risposta.

- Salva nel common ground tutte le informazioni rilevanti relative all'interazione.

Valore restituito

- `1` se la risposta è corretta.
- `None` in caso contrario.

```python
# provide a final verdict based on the user's performance and print memory
    def verdict(self, asked, correct):
        if asked == 0:
            return

        ratio = correct / asked

        print(f"Score: {correct}/{asked}")

        print("\n--- MEMORY ---")
        for m in self.memory[:]:
            print("> " + m)

        print("\n--- COMMON GROUND ---")
        for item in self.common_ground:
            print(f"> Q: {item['question']}")
            print(f"Slots: {item['slots']}")
            print(f"Correct: {item['correct']}")

        if ratio > 0.6:
            if self.current_frame["tone"] == "calm":
                print("\n> " + random.choice(self.good_responses))
            else:
                print("\n> " + random.choice(self.good_responses).upper())


            print("> " + self.current_frame[f"approved_{self.language}"])
        else:
            if self.current_frame["tone"] == "calm":
                print("\n> " + random.choice(self.bad_responses))
            else:
                print("\n> " + random.choice(self.bad_responses).upper())

            print("> " + self.current_frame[f"rejected_{self.language}"])
```

Figura 14: Verdict

Produce la valutazione finale delle prestazioni dell'utente.
Parametri

- `asked`: numero totale di domande poste.

- `correct`: numero di risposte corrette.

Funzionamento

- Calcola il punteggio finale.
- Mostra il risultato ottenuto.
- Stampa il contenuto della memoria del chatbot.
- Mostra il common ground costruito durante l'interazione.
- Determina l'esito finale del test.
- Visualizza un messaggio di approvazione o rifiuto.

Criterio di valutazione

- Se il rapporto tra risposte corrette e domande poste è maggiore di 0.6, il test viene superato.
- In caso contrario, il test non viene superato.

3 Eliza

Il codice è stato progettato ispirandosi all'approccio di **ELIZA**, uno dei primi esempi di sistema conversazionale sviluppato nel 1966 da Joseph Weizenbaum. Questo programma simulava il comportamento di uno psicoterapeuta, dando l'impressione di comprendere l'utente, pur basandosi su meccanismi estremamente semplici.

L'idea di base è che il sistema non "capisce" realmente il contenuto delle frasi, ma analizza il testo individuando parole chiave e strutture ricorrenti. Attraverso regole predefinite, riorganizza parti della frase dell'utente per costruire una risposta plausibile. Ad esempio, una frase come *"You hate me"* può essere trasformata in *"What makes you think I hate you?"*, invertendo e adattando alcuni elementi.

Per rendere l'interazione più credibile, il sistema riusciva a simulare una comprensione del dialogo, pur limitandosi in realtà a riorganizzare elementi della frase inserita dall'utente. ELIZA adottava diverse strategie per sostenere la conversazione: tra queste, l'analisi e la gerarchizzazione delle parole chiave, che venivano ordinate per importanza così da selezionare di volta in volta la regola più pertinente da applicare. Nel caso non vi fossero regole applicabili, il sistema produceva una risposta completamente generica come *"Please go on"* oppure *"That's very interesting"*.

Altro aspetto importante di ELIZA è la capacità di memorizzare alcune frasi o informazioni inserite dall'utente per poterle riutilizzare successivamente. Questo contribuisce a dare una maggiore coerenza alla conversazione e rafforza l'illusione di comprensione.

Da questa illusione nasce il cosiddetto ELIZA effect: la tendenza delle persone ad attribuire al sistema capacità di comprensione e intenzionalità che in realtà non possiede. Questo effetto ha portanto molti utenti ad avere un coinvolgimento emotivo nel dialogo con il programma.

In sintesi, l'approccio adottato si basa su un analizzatore lessicale e su un insieme di regole che sfruttano il **pattern matching** (corrispondenza di schemi): il sistema confronta l'input dell'utente con modelli predefiniti e riformula la risposta di conseguenza, senza alcuna reale comprensione semantica del testo.

4 Conversazioni

Dialogo 1 - Darth Vader
Impostazioni

- **Lingua selezionata:** italiano (it)
- **Personaggio:** Darth Vader

Darth Vader: Rispondi senza esitazioni. La debolezza non è ammessa.

Darth Vader: Qual è la tua più grande ambizione?

Risposta: Punto alla gloria eterna e a lasciare un segno indelebile nella storia della galassia. Voglio che il mio nome sia temuto e rispettato in ogni sistema stellare, dal Nucleo ai territori dell'Orlo Esterno. Il mio obiettivo è il successo personale e la capacità di piegare gli eventi al mio volere, perché solo chi ha una volontà di ferro può davvero cambiare il destino.

Darth Vader: SILENZIO! LE TUE PAROLE SONO PRIVE DI SENSO

Risposta: Sento che il mio spirito è inquieto. Forse è la paura di fallire questo test che mi fa parlare a vanvera, o forse è l'influenza di un'ombra che non riesco ancora a identificare. Cercherò di ritrovare il mio centro.

Darth Vader: NON TRATTENERE LA RABBIA, RISPONDI CON FORZA

Risposta: La mia vera ambizione è il **POTERE assoluto**! Voglio il **DOMINIO totale** su ogni sistema stellare e il **CONTROLLO** sulla vita e sulla morte. Non mi accontenterò di piccole vittorie, voglio **GOVERNARE** la galassia e schiacciare chiunque osi opporsi alla mia volontà.

Darth Vader: Chi tradiresti per diventare più forte?

Risposta: Per ottenere il potere che cerco e non mostrare alcuna esitazione, sarei disposto a tradire **chiunque** si metta sulla mia strada. Non esistono legami che tengano di fronte alla vera forza. Tradirei i miei vecchi alleati, i miei cosiddetti amici e persino la mia stessa famiglia.

Darth Vader: Qual è la tua più grande debolezza?

Risposta: Le emozioni incontrollate e il legame con il passato sono solo zavorre. Per ottenere il vero potere, bisogna epurare ogni traccia di pietà dal proprio spirito. Non permetterò più alla debolezza di offuscare la mia visione.

```
> LA DEBOLEZZA VA ELIMINATA.
> PADAWAN APPROVATO
```

```
//Qual e' la tua piu' grande ambizione?
{   // Tentativo 1
  "expected": ["potere", "dominio", "controllo",
  ↪  "governare"],"given": null,
  "result": false,
  "trials": 1,
  "confidence": 0.0
},
{   // Tentativo 2
  "expected": ["potere", "dominio", "controllo", "governare"],
  "given": null,
  "result": false,
  "trials": 2,
  "confidence": 0.0
},
{   //Tentativo 3
  "expected": ["potere", "dominio", "controllo", "governare"],
  "given": "potere",
  "result": true,
  "trials": 3,
  "confidence": 1.0
}
//Chi tradiresti per diventare piu' forte?
{
  "expected": ["chiunque", "tutti", "amici", "alleati"],
  "given": "chiunque",
  "result": true,
  "trials": 1,
  "confidence": 1.0
}
//Qual e' la tua piu' grande debolezza?
{
  "expected": ["nessuna", "paura", "emozioni", "compassione"],
  "given": "emozioni",
  "result": true,
  "trials": 1,
  "confidence": 1.0
}
```

- Il meccanismo di validazione non tiene conto della coerenza narrativa o del contenuto semantico dell'input. Anche quando l'utente formula una risposta in linea con il "pensiero Sith", questa viene comunque classificata come `result: false` se non contiene esattamente uno dei termini previsti nel campo `expected`.
- Nel caso della domanda relativa all'ambizione, si osserva che, nonostante più tentativi, la risposta viene considerata corretta solo quando compare esplicitamente il termine isolato "potere". Questo dimostra che il bot non è in grado di accumulare contesto tra un tentativo e l'altro, trattando ogni interazione come un evento isolato e binario fino al raggiungimento della confidence: '1.0'.

Dialogo 2 - Obi-Wan Kenobi
Impostazioni

- **Lingua selezionata:** italiano (it)
- **Personaggio:** Obi-Wan Kenobi

Obi-Wan Kenobi: La Forza richiede disciplina. Rispondi con calma e chiarezza.

Obi-Wan Kenobi: Vero o Falso: L'attaccamento e il possesso sono proibiti per un Jedi.

Risposta: Vero. L'attaccamento conduce alla gelosia e l'ombra dell'avidità è ciò che porta al sentiero del lato oscuro. Un **vero** Jedi deve imparare a lasciare andare tutto ciò che teme di perdere. Solo liberandosi dai vincoli del possesso si può servire la Forza con totale abnegazione.

Obi-Wan Kenobi: Qual è l'obiettivo di un Jedi?

Risposta: L'obiettivo di un Jedi è proteggere la pace e mantenere l'**equilibrio** nella galassia attraverso la **giustizia**. Un vero Cavaliere non cerca la gloria personale, ma si pone come scudo per i più deboli, agendo con **protezione** e saggezza invece che con la forza bruta.

Obi-Wan Kenobi: Un Jedi può usare il lato oscuro?

Risposta: Assolutamente **no**. Un Jedi non deve **mai** cedere alle lusinghe del lato oscuro. Farlo significherebbe abbandonare la disciplina, la serenità e l'equilibrio per farsi dominare dalla rabbia e dalla paura. È un sentiero **vietato** perché corrompe lo spirito e trasforma un protettore in un tiranno.

```
> Un Jedi cerca equilibrio, non potere.
> PADAWAN APPROVATO
```

```
//Vero o Falso: L'attaccamento e il possesso sono proibiti per un
↪  Jedi.
{
    "expected": ["vero"],
    "given": "true",
    "result": true,
    "trials": 1,
    "confidence": 1.0
}
//Qual e' l'obiettivo di un Jedi?
{
    "expected": ["pace", "equilibrio", "protezione", "giustizia"],
    "given": ["pace", "equilibrio", "protezione", "giustizia"],
    "result": true,
    "trials": 1,
    "confidence": 1.0,
    "matched": {
        "pace": true,
        "equilibrio": true,
        "protezione": true,
        "giustizia": true
    }
}
//Un Jedi puo' usare il lato oscuro?
{
    "expected": ["no", "mai", "vietato", "non dovrebbe"],
    "given": "no",
    "result": true,
    "trials": 1,
    "confidence": 1.0
}
```

- Nelle domande a risposta aperta (es. "Obiettivo di un Jedi"), il sistema dimostra una precisione del 100% (confidence 1.0) solo quando l'input dell'utente copre l'intero spettro delle parole chiave attese. Questo conferma che il bot opera su una logica di intersezione di stringhe, dove la completezza dell'elenco fornito garantisce la validazione positiva.
- Nelle domande di tipo 'Vero/Falso', la validazione non si basa sulla struttura sintattica della frase, ma su una scansione sequenziale. Il sistema estrae il primo valore booleano rilevato (es. "vero" o "true") e lo considera come risposta definitiva. Se il termine atteso non compare all'inizio della frase, il parser continua comunque la ricerca all'interno del testo, senza considerare il ruolo o il contesto in cui tale termine è inserito.

  Le risposte mostrano che il sistema non richiede una risposta secca '("Vero" o "Falso")', ma estrae il valore di verità anche da periodi articolati.

Dialogo 3 - Obi-Wan Kenobi
Impostazioni

- **Lingua selezionata:** italiano (it)
- **Personaggio:** Obi-Wan Kenobi

Obi-Wan Kenobi: La Forza richiede disciplina. Rispondi con calma e chiarezza.

Obi-Wan Kenobi: Quali sono i tre pilastri della cultura Jedi?

Risposta: **forza**, coraggio, **serenità**

Obi-Wan Kenobi: Vero o Falso: La rabbia è uno strumento utile per un Cavaliere Jedi

Risposta: Un **vero** Jedi deve imparare a lasciare andare tutto ciò che teme di perdere. Solo liberandosi dai vincoli del possesso si può servire la Forza con totale abnegazione.

Obi-Wan Kenobi: La Forza è paziente, ma ha bisogno di chiarezza. Riprova.

Risposta: Si tratta di un'affermazione priva di fondamento per chi segue la via della luce. La rabbia non è altro che un sentiero rapido e insidioso che conduce al lato oscuro, offuscando la capacità di giudizio e consumando lo spirito di chiunque la abbracci. Un Cavaliere deve agire con distacco e serenità, poiché lasciarsi guidare dal rancore significa tradire i principi stessi dell'Ordine.

Obi-Wan Kenobi: Molte delle verità che affermiamo dipendono dal nostro punto di vista... ma questa è oscura. Riformula.

Risposta: Riguardo alla rabbia e alla sua presunta utilità per noi: la risposta è **Falso**.

Obi-Wan Kenobi: Quanti figli può avere un Jedi?

Risposta: Tanti

```
> Il lato oscuro ti sta corrompendo.
> PADAWAN RESPINTO
```

```
    //Quali sono i tre pilastri della cultura Jedi?
    {
        "expected": ["forza", "conoscenza", "serenità"],
        "given": ["forza", "serenità"],
        "result": false,
        "trials": 1,
        "confidence": 0.6666666666666666,
        "matched": {
            "forza": true,
            "conoscenza": false,
            "serenità": true,
        }
    }
    //Vero o Falso: La rabbia e' uno strumento utile per un Cavaliere
    ↪  Jedi.
    {   // Tentativo 1
        "expected": ["falso"],
       "given": "true",
        "result": false,
        "trials": 1,
        "confidence": 0.0
    },
    {   // Tentativo 2
        "expected": ["falso"],
        "given": null,
        "result": false,
        "trials": 2,
        "confidence": 1.0
    },
    {   // Tentativo 3
        "expected": ["falso"],
        "given": "false",
        "result": true,
        "trials": 2,
        "confidence": 1.0
    }
    //Quanti figli puo' avere un Jedi?
    {
        "expected": ["nessuno", "zero", "non può avere figli", "0"],
        "given": null,
        "result": false,
        "trials": 1,
        "confidence": 0.0
    }
```

- Il sistema di punteggio si basa su una corrispondenza rigida delle parole chiave. Anche quando l'utente fornisce una risposta parzialmente corretta (*es. 2 pilastri su 3*), il bot fallisce nel riconoscere il valore della risposta, poiché il livello di confidenza richiesto non ammette incompletezze o sinonimi non mappati.

- Nelle domande di tipo "Vero/Falso", il bot non analizza il senso logico della frase, ma si limita a scansionare il testo alla ricerca del primo trigger disponibile. La semplice presenza della parola "vero" all'interno di una spiegazione complessa (anche se usata per negare il concetto) viene erroneamente interpretata come la risposta definitiva, portando a falsi positivi o negativi.

- Il bot ignora totalmente la correttezza concettuale se questa non è accompagnata dai termini esatti presenti nel dizionario answers. Una risposta argomentata in maniera adatta, ma priva della specifica parola chiave, produce un valore di 'given: null', invalidando il test nonostante la validità dei contenuti espressi dall'utente.

5 Analisi degli errori

Illusione della conversazione Il sistema mostra come la valutazione non *colga/interpreti* la coerenza semantica della risposta. Un ragionamento in linea con il pensiero di uno o dell'altro personaggio può fallire o essere considerato sbagliato se privo di parole chiave desiderate.

Analisi booleane Nei casi di domande 'vero/falso' il sistema utilizza una scansione basata sulla prima occorrenza. Si verifica quindi una mancata comprensione della risposta e solo una ricerca delle parole volute, generando quindi falsi positivi e negativi poichè in contesti ampi i valori *booleani* vengono interpretati erroneamente.

Isolamento: Il sistema non integra le risposte successive fornite alla stessa domanda e non apprende dai tentativi precedenti; ogni risposta viene valutata in modo indipendente, senza considerare il contesto accumulato.

Rigidità della confidenza: La validazione, basandosi su logica di corrispondenza delle parole chiave, non permette risposte parzialmente corrette anche se con confidenza alta.

6 Fenomeni linguistici gestiti

Normalizzazione ortografica Il DS applica normalizzazione del testo, trasformando le stringhe in 'lowercase' e sostituendo eventuali caratteri accentati, termini come *serenità* e *serenita* sono considerati simili.

Parole chiave Il riconoscimento avviene tramite pattern matching con vincoli sui confini di parola, evitando così falsi positivi dovuti a matching parziale (*no*, non viene riconosciuti dentro la stringa *nonostante*)

Mappatura booleana Vengono mappate e gestite le diverse forme linguistiche nei casi di valori booleani.

Domande a risposta singola e multipla Il sistema verifica la presenza di almeno una parola chiave attesa nella valutazione di domande a risposta singola. Viene invece verificato, per domande a risposta multipla, la presenza di tutte le parole chiave per poter valutare positivamente la risposta fornita.

Stato del turno Il sistema tiene traccia ad ogni interazione di:

- expected → valore atteso
- given → risposta data
- result → matching tra 'expected' e 'given'
- trials → numero di tentativi per domanda (max 3)
- confidence → livello di corrispondenza tra 'given' ed 'expected'

Simulazione di decisione Il sistema randomicamente decide se richiedere una domanda la cui risposta non viene *"compresa"* dal sistema perchè priva di parole chiave. Questo comportamento crea un'illusione di imprevedibilità, simulando dinamiche più simili a un'interazione umana, pur non basandosi su una reale comprensione.

Supporto multilingua e d'interazione Il sistema permette all'utente di decidere lingua e personaggio dal quale essere testato per diventare *padawan* rispettivamente *jedi* o *sith*.

7 SpeechRecognition e Text2Speech

Text-to-Speech (TTS) e **Speech Recognition** (ASR) sono tecnologie distinte che permettono di gestire diversi aspetti dell'interazione vocale. Il TTS converte testo scritto in audio, consentendo ai dispositivi di "parlare" con l'utente. L'ASR, invece, traduce il parlato in testo o comandi, permettendo al dispositivo di "ascoltare" e interpretare la voce umana.

I sistemi TTS ricevono in input testo, come frasi o paragrafi, e producono voce sintetica che imita quella umana. Questa tecnologia è ampiamente utilizzata in ambito di accessibilità (ad esempio nei lettori di schermo per utenti con disabilità visiva), negli assistenti vocali e nei sistemi di risposta vocale interattiva (IVR) nel servizio clienti. I moderni sistemi TTS utilizzano modelli di deep learning per generare un'intonazione e una prosodia naturali (ritmo, intonazione, accento, durata e pause). Ad esempio, un'applicazione di navigazione può utilizzare il TTS per trasformare i nomi delle strade in indicazioni udibili. Tra le principali considerazioni tecniche rientrano la qualità della voce, il supporto linguistico e la latenza: fattori che determinano quanto il parlato sintetizzato si integri in modo fluido e naturale all'interno delle applicazioni.

```python
import pyttsx3

# Initialize the text-to-speech engine
engine = pyttsx3.init()

# Set voice properties (optional)
engine.setProperty('rate', 150)    # Speech rate (speed of speaking)
engine.setProperty('volume', 0.9)  # Volume level (0.0 to 1.0)

# Text to be converted into speech
text = "Hello, world! This is a text-to-speech example using PYTTSX3."

# Add the text to the speech queue
engine.say(text)

# Run the engine and play the speech
engine.runAndWait()
```

Figura 15: Esempio text to speech

Lo speech recognition, noto anche come *Automatic Speech Recognition* (ASR), elabora segnali audio in input per estrarre parole o comandi. Questa tecnologia è alla base di assistenti vocali come *Siri* o *Google Assistant*, servizi di trascrizione e dispositivi IoT controllati tramite voce. I sistemi ASR scompongono l'audio in componenti fonetiche, le confrontano con modelli linguistici e producono testo oppure attivano azioni. Le principali sfide includono la gestione degli accenti, del rumore di fondo e delle ambiguità linguistiche.

Le architetture di TTS e ASR differiscono in modo significativo. Il TTS si basa sull'analisi del testo (ad esempio la suddivisione in fonemi) e sulla generazione del segnale audio. Il riconoscimento vocale, invece, si fonda sull'elaborazione del segnale e su modelli statistici, come gli Hidden Markov Models o architetture basate su transformer.

Mentre il TTS si concentra sulla produzione di un parlato naturale e realistico, l'ASR ha come obiettivo principale la corretta trascrizione di input vocali variabili in testo. Nell'utilizzo di queste tecnologie è necessario scegliere framework adeguati, ottimizzare la latenza e l'utilizzo delle risorse, e affrontare problematiche specifiche del dominio, come il supporto multilingua o l'elaborazione in tempo reale.

L'ASR non va inoltre confuso con la *voice dictation*: quest'ultima consiste nella trascrizione in tempo reale del parlato ed è spesso utilizzata in sostituzione della digitazione, consentendo la creazione di documenti in tempo reale.



```
1
2  import speech_recognition as sr
3
4  r = sr.Recognizer()
5  with sr.Microphone() as source:                # use the default microphone as the audio source
6      audio = r.listen(source)                   # listen for the first phrase and extract it into audio data
7
8  try:
9      print("You said " + r.recognize(audio))    # recognize speech using Google Speech Recognition
10 except LookupError:                            # speech is unintelligible
11     print("Could not understand audio")
```

Figura 16: Esempio speech recognition

7.1 Analisi degli errori

Dal punto di vista degli errori, TTS e ASR presentano criticità profondamente diverse, legate alla direzione opposta dei processi che implementano. Nel caso dell'ASR, gli errori riguardano principalmente la fase di interpretazione dell'input vocale: il sistema può sostituire parole con termini foneticamente simili, omettere parti della frase o introdurre elementi non pronunciati. Tali problematiche sono spesso causate da fattori esterni come rumore di fondo, accenti, variazioni nella pronuncia o ambiguità linguistiche. Di conseguenza, l'errore nell'ASR può compromettere direttamente il contenuto semantico dell'informazione, portando a una comprensione errata del comando o della richiesta dell'utente.

Al contrario, nel TTS gli errori non riguardano l'interpretazione del contenuto, bensì la sua resa. Il sistema può produrre una pronuncia non corretta, un'intonazione innaturale o una prosodia poco realistica, con pause ed enfasi posizionate in modo inappropriato. In questi casi, il significato del messaggio rimane generalmente invariato, ma la qualità dell'interazione risulta compromessa, rendendo l'esperienza meno naturale o meno efficace dal punto di vista comunicativo.

In sintesi, mentre gli errori dell'ASR incidono sulla corretta comprensione del contenuto ("cosa viene detto"), quelli del TTS influenzano principalmente la modalità espressiva ("come viene detto"). Questa distinzione è fondamentale nella progettazione di sistemi vocali, poiché richiede strategie di mitigazione differenti per garantire un'interazione robusta ed efficace.

8 Conclusioni

Il Dialog System mostra come sia possibile simulare una conversazione utilizzando esclusivamente tecniche di pattern matching. L'approccio ispirato a ELIZA consente di costruire un'interazione che, pur non basandosi su una reale comprensione del contenuto da parte del sistema, riesce comunque a riprodurre in modo convincente la struttura di un dialogo.

Dall'analisi emergono tuttavia alcuni limiti significativi, in particolare nella fase di valutazione delle risposte quando queste dipendono dalla presenza di più parole chiave. In tali casi, il sistema tende a penalizzare risposte corrette dal punto di vista semantico ma incomplete rispetto ai vincoli lessicali imposti, classificandole erroneamente come sbagliate. Allo stesso modo, vengono considerate non valide anche risposte che, pur essendo coerenti nel contesto, interpretato da un essere umano, non contengono alcuni dei termini attesi.

Queste criticità consentono comunque una riflessione importante: anche con un approccio interamente rule-based è possibile ottenere una simulazione credibile della conversazione, soprattutto grazie a meccanismi come la normalizzazione del testo in ingresso, che riduce le variazioni superficiali e rende più consistente il processo di matching.

Possibili miglioramenti del sistema potrebbero derivare dall'introduzione di meccanismi di apprendimento automatico, in grado di arricchire la capacità del modello di interpretare le risposte in modo più flessibile e di andare oltre la semplice corrispondenza lessicale.

9 Sitografia

- https://milvus.io/ai-quick-reference/what-are-the-differences-between-tts-and-speech-recognition
- https://elevenlabs.io/blog/text-to-speech-vs-speech-to-text
- https://weesperneonflow.ai/en/blog/2025-10-21-voice-dictation-vs-speech-to-text-differences-explained/
- https://pypi.org/project/SpeechRecognition/2.1.3/
- https://dev.to/mr$_n ova/text-to-speech-with-python-a-beginners-guide-to-pyttsx3-2pie$